

TPP-DLW Toolpath Generator

New User Guide for STL-to-TXT Conversion

This guide explains how to convert an STL file into the tab-separated segment text format used by the LabVIEW writing interface. It focuses on the fastest current workflow and the checks that prevent false toolpath lines.

Recommended path: use `stl_slice_export_mm_woodpile_true_resample.m` for large STL height maps. Use the modular `tpddl_w_process` pipeline for general STL/STEP workflows or the GUI.

1. Project Layout

Fast converter	<code>stl_slice_export_mm_woodpile_true_resample.m</code> - optimized STL height-map to TXT exporter.
General pipeline	<code>tpddl_w_process.m</code> plus core/ helpers for STL/STEP imports and arbitrary scan angles.
Preview tools	<code>viz/preview_toolpath.m</code> , <code>viz/preview_3d.m</code> , and layer comparison tools.
Examples/tests	<code>examples/</code> and <code>tests/</code> show basic usage and expected output format.
Generated data	<code>output/</code> , STL files, and large TXT exports are ignored by Git by default.

2. Quick Start

From MATLAB, open the project folder and run:

```
cd('/path/to/3D_Printer_construct')
run('stl_slice_export_mm_woodpile_true_resample.m')
```

- Set `STLPath` to your input STL file.
- Set `OutTxt` to the TXT file you want to create.
- Start with coarse `XYPitch` and `DZ` for preview runs, then restore final values for production.
- Read the console summary. Warnings about skipped odd scanlines usually mean the STL has open or non-manifold contours.

3. Output Format

The output TXT is tab-separated, with one row per laser-on scan segment or Z transition:

```
x1    Y1    Z1    x2    Y2    Z2
```

- Rows where `Z1 == Z2` are scan/write segments.
- Rows where `Z1 != Z2` are layer-to-layer Z moves.
- Coordinates are in millimeters.
- The modular pipeline negates Z for stage convention; the standalone exporter can write physical Z or layer indices depending on `ZAsIndex`.

4. Main Parameters

Parameter	Purpose	Typical use
STLPath	Input STL file to convert.	Use a watertight STL when possible.
OutTxt	Destination tab-separated TXT file.	Use output/name.txt for generated files.
TargetMaxXY	Scales the model to a target XY footprint in mm.	Scalar for square fit, or [maxX maxY].
XPitch	Spacing between adjacent raster scanlines.	Larger for preview, final value for production.
DZ	Layer thickness / vertical slicing step.	Larger for preview, final value for production.
SquareGrid	Forces equal X/Y grid counts.	Recommended for woodpile-style output.
WoodpileMode	Alternates horizontal and vertical writing by layer.	Keep true for orthogonal resampling.
Serpentine	Alternates scan direction to reduce travel.	Usually true.
OptimizePath	Greedy nearest-neighbor segment ordering.	Can be expensive; capped by OptimizeMaxSegments.
OptimizeMaxSegments	Skips greedy ordering above this per-layer row count.	Default 15000; lower it for speed.
TraceContour	Writes boundary outline segments before hatch fill.	Keep false for height-map raster exports.
CoordMode	Uses grid centers or voxel edges for segment endpoints.	Use centers unless your printer ignores point-like writes.
OutputSignificantDigits	Controls compact TXT numeric precision.	Default 6 keeps values like 0.0002 readable.
Tol	Geometric tolerance in mm.	Default is usually fine.

5. Preview and Quality Checks

After generating a TXT file, preview a layer before printing:

```
addpath(genpath(pwd))
segments = load('output/Final.txt');
preview_toolpath(segments, 'Layer', 1, 'Mode', '2d')
preview_3d(segments, 'EveryN', 2)
```

Before sending a file to the printer, check for long strokes that cross empty space, out-of-bounds coordinates, unexpected zero-length rows, and unusually high segment counts on one layer.

If the first rows are short fragments along a boundary such as $X = 0$, contour tracing is enabled. Keep TraceContour and ContourFirst false for height-map raster exports so the file starts with hatch-fill rows.

Important: the current converter skips unresolved odd scanlines instead of force-pairing them. This prevents false long write lines. If many scanlines are skipped, repair the STL or use a height-map raster workflow.

6. Troubleshooting

Long lines across empty regions	Usually caused by open/non-manifold STL contours. The safe behavior is to skip odd scanlines and warn. Repair the STL or use height-map rasterization.
Very slow conversion	Increase XYPitch and DZ for preview, lower OptimizeMaxSegments, or disable greedy path optimization.
Missing small features	Check whether XYPitch or DZ is too coarse. Also check for skipped odd scanlines.
TXT is huge	This is expected for fine pitch and large footprints. Keep generated TXT files under output/ so Git ignores them.
STL import fails	Confirm the STL is binary or valid ASCII, not truncated, and that the file path is correct.

7. Recommended Workflow

- **Preview pass:** use coarse pitch and layer height; generate quickly and inspect several layers.
- **Geometry check:** look for warnings about skipped odd scanlines or open contours.
- **Final pass:** restore production XYPitch and DZ; regenerate TXT.
- **Visual QA:** preview first, middle, and last layers; check that no scanline crosses empty space.
- **Archive:** commit code/config changes, not large generated files. Keep STL/TXT outputs ignored unless you intentionally need to version them.

8. Quick Checklist Before Printing

[] Input STL is correct	Right geometry, right units, expected bounding box.
[] Parameters reviewed	Target size, XY pitch, DZ, coordinate mode, and output path are intentional.
[] Console warnings reviewed	No unexpected truncated STL, odd scanline, or out-of-bounds warnings.
[] Layers previewed	First, middle, and final layers look physically plausible.
[] Output format checked	TXT has six numeric columns and the expected number of layers.
[] Generated files managed	Large STL/TXT files are kept out of Git unless deliberately tracked.

For maintainers: keep the false-line prevention behavior conservative. Skipping an unresolved scanline is preferable to writing a long segment across empty space.